Aula 5 — Maestro Avançado + Seminário Técnico

Seletores, esperas, suíte organizada — e a última entrega do curso

Prof. Jackson Smith Moisés Matias
PUC Minas IEC · 22/10/2026

Recap da Aula 4

- **Setup:** emulador Android + Maestro + CineFav rodando no *seu* computador
- **Maestro:** flow em YAML — `launchApp`, `tapOn`, `inputText`, `assertVisible`
- `id` **(testID)** em vez de texto; **fragmento** `_fragments/login.yaml` pra não repetir login
- **Lab Mobile:** flows `02-search` e `03-favorite` + Jest

> **Vence hoje (22/10):** Lab Mobile e Lab Playwright PWA. Hoje abrimos um **novo lab** (Arquitetura de Testes) e reservamos 10 min pras dúvidas do Lab Mobile.

Agenda da aula (3h30)

- **35min — Maestro avançado:** hierarquia de seletores, acessibilidade, esperas sem `sleep`
- **40min — Arquitetura de flows:** fragmentos, `env`, POM — e **refatoramos um flow juntos**
- **30min** — Intervalo
- **55min — Lab Arquitetura de Testes** em sala (5 níveis; fecha em casa) · dúvidas do Lab Mobile
- **40min — Seminário Técnico (em grupo)**
- **~10min** — Combinados

> A refatoração do Nível 1 é **só YAML**: quem está sem emulador acompanha e edita junto; roda depois.

Objetivos da aula

- **Escolher** o seletor certo quando o elemento *não tem* `id`
- **Esperar** sem `sleep` e medir flakiness
- **Organizar** flows em camadas: **flow** (história) → **fragmento** (passos) → **POM** (seletores)
- **Reaproveitar** em vez de copiar: fragmento, `env`, Page Object
- **Refatorar** um flow "que funciona mas ninguém lê"
- **Escolher** o tema do **Seminário Técnico** em grupo

Parte 1 — Maestro avançado

35 min · seletor, espera, suíte

A hierarquia de seletores — do mais ao menos estável

#	Seletor	Exemplo	Quebra quando...
1	**`id`** (testID)	`id: "login-submit-button"`	o dev renomeia o testID
2	**Texto / label de acessibilidade**	`text: "Entrar"`	muda a tradução ou o texto
3	**Relativo** (`below`, `childOf`...)	`below: "Matrix"`	muda o layout
4	**`index`**	`text: "Favoritar"` + `index: 1`	muda a ordem da lista
5	**Ponto** (`point: 50%,80%`)	`point: "90%,10%"`	muda o tamanho da tela

> **Regra:** desça um degrau só quando o de cima não existe. Se você chegou no 5, **peça um `testID` pro dev** — é a correção de verdade.

De onde vem o `id`? (e a acessibilidade)

No React Native, **uma prop vira duas coisas**:

Prop no componente	Android	Maestro acha por
`testID="login-submit-button"`	`resource-id`	`id:`
`accessibilityLabel="Entrar"`	`content-desc`	`text:`
`accessibilityRole="button"`	`class` / role	— (não é seletor)

- `text:` do Maestro casa **texto visível *ou* label de acessibilidade**
- Um app bem acessível é **mais fácil de testar**: o label serve ao leitor de tela e ao seu flow
- **Pergunta do Hulgo:** "e o `class`?" — evite como seletor: `android.widget.TextView` aparece em dezenas de lugares

> O `id` garante que o elemento **existe**; o label/role garante que é **usável** por quem depende de leitor de tela.

Elemento sem `id` — três saídas, em ordem

```yaml
# 1) Texto (aceita regex) — ok se o texto é estável
- tapOn: "Matrix"
- tapOn: ".*Matri.*"

# 2) Relativo a um vizinho que TEM id/texto
- tapOn:
    text: "Favoritar"
    below: "Matrix"

# 3) Filho de um container com id
- tapOn:
    text: "Favoritar"
    childOf:
      id: "movie-card-603"
```

`index: 2` e `point: "50%,50%"` funcionam hoje e **quebram na primeira mudança de lista ou de tela**. Último recurso.

Achando o seletor — `maestro hierarchy`

Com o app aberto no emulador:

```
maestro hierarchy
```

Imprime a **árvore de elementos da tela atual** (JSON): `resource-id`, `text`, `accessibilityText`, `bounds`, `enabled`…

- Procure o **elemento que você quer tocar** e copie o `resource-id` → vira `id:`
- Não tem `resource-id`? veja o `text` / `accessibilityText`
- Sem instalar nada além do Maestro: é o **fallback** quando o Studio não abre

```
maestro hierarchy | grep -i "movie-card"   # filtra pelo que importa
```

> O **Maestro Studio** (app desktop) faz o mesmo de forma visual — opcional. Os testIDs do CineFav também estão em `pratica/src/utils/testIDs.ts`.

Esperar sem `sleep`

O Maestro **já espera um pouco** a tela estabilizar antes de cada comando. Pra o resto:

```yaml
- extendedWaitUntil:          # espera aparecer (timeout em ms)
    visible: { id: "movielist-screen" }
    timeout: 10000

- scrollUntilVisible:         # rola até achar (lista longa)
    element: { text: "Interestelar" }
    direction: DOWN
    timeout: 20000

- waitForAnimationToEnd       # animação em andamento
```

❌ `sleep` / "espera fixa de 5s": ou é lento demais, ou curto demais na máquina do colega. Espere por uma **condição**, nunca por um **tempo**.

Controle de fluxo e navegação

```yaml
- back                          # botão voltar do Android
- hideKeyboard                  # fecha o teclado
- pressKey: Enter

- assertNotVisible:             # o contrário de assertVisible
    id: "favorites-item-603"

- repeat:                       # repete N vezes
    times: 3
    commands:
      - tapOn: "Próxima página"

- retry:                        # tenta de novo se algo falhar (flake transitório)
    maxRetries: 2
    commands:
      - tapOn:
          id: "movielist-search-button"
```

> `retry` é **curativo**: use pra instabilidade externa conhecida, não pra esconder teste mal escrito.

Condicional: popups e plataforma

```yaml
# Dispensa o popup de permissão SE ele aparecer
- runFlow:
    when:
      visible: "Permitir"
    commands:
      - tapOn: "Permitir"

# Passo só em uma plataforma
- runFlow:
    when:
      platform: Android
    commands:
      - back
```

- `when: visible:` **não espera** a tela — olha o que está lá *agora*; sincronize antes (`extendedWaitUntil`)
- Ótimo pra popups de permissão e estados que variam entre execuções

Suíte organizada — `env`, fragmentos e `tags`

```yaml
# flows/02-search.yaml
appId: com.puciec.cinefav
tags:
  - smoke
env:
  SEARCH_TERM: "Matrix"
---
- launchApp
- runFlow: _fragments/login.yaml
- tapOn: { id: "movielist-search-button" }
```

```
maestro test flows/                              # todos os flows da pasta
maestro test --include-tags=smoke flows/     # só os marcados "smoke"
maestro test -e SEARCH_TERM=Avatar flows/02-search.yaml   # sobrescreve o env
```

> **Smoke** = poucos flows rápidos que dizem "o app abre e o essencial funciona". Rode em todo commit; os demais, à noite.

Medindo flakiness (o eliminatório do Lab)

> Flakiness > 10% em 3 execuções seguidas = **−3 pts** no Lab Mobile.

Rode **3 vezes seguidas** antes de entregar:

```
cd exercicios/02-lab-mobile-maestro-jest/pratica
ls flows

for i in 1 2 3; do maestro test flows/02-search.yaml || echo "FALHOU $i"; done
for i in 1 2 3; do maestro test flows/03-favorite.yaml || echo "FALHOU $i"; done
```

- Passou 3/3? ✅ pode entregar
- Falhou uma? veja **onde** (`maestro hierarchy`): quase sempre é espera faltando ou seletor por texto

Parte 2 — Arquitetura de flows

40 min · organizar e reaproveitar — e refatorar juntos

Quando a suíte de flows vira bagunça

Cheiro (*smell*)	Sintoma	Consequência
Duplicação	login copiado em todo flow	muda o login → conserta em N lugares
ID solto	`"login-submit-button"` escrito 40×	renomeou o testID → 40 flows quebram
Passo repetido	favoritar o filme A, depois o B, igual	flow longo, difícil de ler
Flow ilegível	30 linhas de `tapOn` / `inputText`	ninguém sabe **o que** ele prova
Flow gigante	um YAML de 80 linhas	uma falha, nenhum diagnóstico

> Objetivo: o flow deve ler como uma **história do usuário**. Como achar o botão é problema de **outra camada**.

As camadas de uma suíte de flows

Camada	O que contém	No Maestro
1. Flow	a história + as asserções	`flows/05-favoritos-composto.yaml`
2. Fragmento	um pedaço reutilizável de passos (aceita `env`)	`_fragments/login.yaml`, `_fragments/buscar.yaml`
3. Page Object	onde está cada elemento (os ids)	`elements/login.js` (`runScript` + `output`)
4. App	o que está sendo testado	o `.apk`

- Cada camada só conhece a de baixo → mudou o app, **conserta numa camada só**
- Mesma ideia de qualquer suíte organizada: história no topo, detalhes embaixo

Fragmento no Maestro — reaproveitar passos

```yaml
# flows/_fragments/abrir-busca.yaml
appId: com.puciec.cinefav
---
- runFlow: login.yaml              # fragmentos compõem fragmentos
- tapOn: { id: "movielist-search-button" }
- assertVisible: { id: "search-screen" }
```

```yaml
# flows/02-busca.yaml
appId: com.puciec.cinefav
tags: [regressao]
---
- launchApp
- runFlow: _fragments/abrir-busca.yaml    # 1 linha em vez de 8
- tapOn: { id: "search-input" }
- inputText: "Matrix"
```

> ⚠️ O caminho do `runFlow` é **relativo ao arquivo que chama** (dentro de `_fragments/`, o login é só `login.yaml`).

Fragmento parametrizado — `env` no `runFlow`

```yaml
# flows/_fragments/buscar.yaml          # recebe ${TERM}
appId: com.puciec.cinefav
---
- tapOn: { id: "search-input" }
- inputText: ${TERM}
- assertVisible: ${TERM}
```

```yaml
# flows/03-busca-parametrizada.yaml
- runFlow:
    file: _fragments/buscar.yaml
    env:
      TERM: "Matrix"
- tapOn: { id: "search-clear-button" }
- runFlow:
    file: _fragments/buscar.yaml        # o MESMO fragmento, outro dado
    env:
      TERM: "Avatar"
```

> Mesma lógica, dado diferente: um fragmento no lugar de dois flows copiados.

Page Object no Maestro (POM) — `runScript` + `output`

Os **seletores** num lugar só; os flows não têm nenhum `id` escrito:

```
// flows/elements/login.js        (Maestro: não é Node — sem require/export)
output.login = { email: "login-email-input", senha: "login-password-input", entrar: "login-submit-button" };
```

```
- runScript: elements/login.js
- tapOn: { id: ${output.login.email} }
- inputText: "aluno@puc.br"
- tapOn: { id: ${output.login.entrar} }
```

> Renomeou o `testID` ? Muda **1 linha** em `elements/login.js` — os 5 flows continuam verdes.

Fragmento × fragmento + `env` × POM — quando usar cada um

Ferramenta	Reaproveita…	Exemplo	Use quando
Fragmento	uma **sequência de passos**	login, abrir busca	o mesmo caminho aparece em vários flows
Fragmento + `env`	a sequência com **dados diferentes**	`favoritar` `${MOVIE_ID}`, `buscar` `${TERM}`	muda só o dado
POM (`elements/*.js`)	os **seletores**	`output.login.email`	muitos flows tocam os mesmos elementos

> Não exagere: com 2 flows e 1 tela, **abstração demais** é pior que duplicação. Extraia na **3ª repetição**.

🔧 Refatorando juntos — Nível 1 (~25 min, só YAML)

Abra `flows/00-refatorar.yaml` — **ele passa, mas está cheio de cheiros**.

```
cd exercicios/05-lab-arquitetura-testes/pratica
ls flows                                    # confirma o lugar certo (Windows: dir flows)
maestro test flows/00-refatorar.yaml        # com emulador: verde antes de mexer
```

1. Troque o login inteiro (6 passos) por **uma linha:** `runFlow: _fragments/login.yaml`
2. Crie `_fragments/favoritar.yaml` (`tapOn` no `movie-card-heart-${MOVIE_ID}`)
3. Chame o fragmento **2×**, com `env: MOVIE_ID: "603"` e `"27205"`
4. Rode de novo — **continua verde**: mesmo comportamento, leitura diferente

> **Régua:** no `00-refatorar.yaml` , nenhum id de login e nenhum `movie-card-heart-…` escrito. Sem emulador? Edite junto; rode depois.

Recap — Maestro avançado + arquitetura

1. **`id` > texto > relativo > index > ponto** — desça um degrau só se precisar
2. **`maestro hierarchy`** descobre o seletor; não chute
3. **Espere por condição** (`extendedWaitUntil`, `scrollUntilVisible`), nunca por tempo
4. **Camadas:** flow (história) → fragmento (passos) → POM (seletores)
5. **Fragmento + `env`** reaproveita passos; **POM** reaproveita seletores
6. **Rode 3×** antes de entregar — e extraia na 3ª repetição, não na 1ª

☕ Intervalo

30 minutos

Parte 3 — Lab Arquitetura de Testes

55 min · 5 níveis, junto com a turma

O Lab — 5 níveis, dificuldade crescente

Nível	O quê	Onde
1	refatorar `00-refatorar` + fragmento `favoritar` *(já fizemos)*	antes do intervalo
2	fragmento `abrir-busca` + flow `02-busca`	em aula
3	fragmento **parametrizado** (`TERM`) chamado 2×	em aula
4 🔴	**POM** (`elements/*.js`) + `04-pom-login`	casa
5 🔴🔴	`05-favoritos-composto` : tudo junto + `assertNotVisible`	casa

Sem emulador? Escreva o YAML agora e rode depois. **Dupla** vale pra desbloquear setup; a entrega é individual.
Antes: **10 min** pras dúvidas do **Lab Mobile** (vence hoje).

Travou? Os suspeitos de sempre

Sintoma	Causa provável	Conserto
"Maestro Android driver did not start up in time"	emulador lento / pouca RAM	feche programas, reinicie o `adb`
`tapOn` não acha o elemento	seletor errado ou tela ainda carregando	`maestro hierarchy` + `extendedWaitUntil`
`inputText` digita no lugar errado	campo sem foco	`tapOn` no campo **antes**
`assertVisible` falha em texto que "está lá"	texto parcial / maiúscula	use regex: `".*Matrix.*"`
Fragmento não encontrado	caminho errado	caminho **relativo ao flow** (`_fragments/login.yaml`)
Passa 1 vez, falha na 2ª	estado do app sobrando	`launchApp` com `clearState: true`

Como tocar o Lab — e como saber que acertou

```
cd exercicios/05-lab-arquitetura-testes/pratica
ls flows
maestro test flows/01-smoke.yaml                    # modelo (deve passar)
maestro test flows/02-busca.yaml                    # o seu, nível a nível
maestro test --include-tags=regressao flows/        # todos os flows de regressão
```

- **Modelos prontos** 🟦: `_fragments/login.yaml` e `01-smoke.yaml` — leia antes de escrever
- **Rode 3×** o que você escreveu: o lab pune flake (−3 pts)
- **Entrega:** PR com `RESPOSTAS.md` (4 perguntas) + print/vídeo dos flows no Android
- **Bônus sem nota:** relatório JUnit (`--format junit`)

Hoje em aula × em casa

	Em aula (hoje)	Em casa (fecha o lab)
Nível 1	refatorar `00-refatorar` **juntos** (antes do intervalo)	—
Níveis 2 e 3	fragmento `abrir-busca` e fragmento parametrizado (começar; quem tem emulador roda)	terminar e rodar 3×
Níveis 4 e 5	POM e flow composto: demonstrados pelo professor	fazer (🔴 desafio)
Entrega	tirar dúvidas; combinar tema do Seminário	`RESPOSTAS.md` + print/vídeo no Android + PR

> Meta realista de hoje: **nível 1 pronto** e os níveis 2 e 3 começados. O resto é pra casa — **rode 3×** antes de entregar.

Parte 4 — Seminário Técnico

40 min · o que é, temas, rubrica

O Seminário Técnico (Projeto Final) em uma frase

Seminário técnico = cada grupo escolhe **um tema de testes** atual (agentes de IA, shift-right, web/PWA, mobile, pipeline, acessibilidade...), **constrói um artefato pequeno e demonstrável**, e **apresenta** o que descobriu.

- **Grupo de 3 a 4 pessoas** — vocês se organizam; **todos** apresentam
- **Apresentação curta e técnica** — não é só slide, tem **demo** do que o grupo fez
- **Relatório curto** — o que fez, o que achou, o que não funcionou
- Tudo com ferramenta **gratuita** e emulador/navegador — sem cartão de crédito, sem serviço pago

> **Datas:** o grupo escolhe o tema **hoje** · apresenta em **29/10** (Aula 6, última aula) · entrega o relatório e o repositório conforme o **Canvas**.

Entregáveis

#	Entregável	O que é
1	**Artefato técnico**	repositório (ou pasta no fork) com o que o grupo construiu/rodou, README com **comandos exatos**
2	**Apresentação**	~10 min: problema → ferramenta → **demo** → achado → limitação, + 5 min de Q&A
3	**Relatório**	2 páginas: o que fez, evidências (prints/saída), o que **não** funcionou, conclusão

- **Reprodutibilidade:** outra pessoa roda seu README e chega ao mesmo resultado
- **Contribuição individual:** o histórico de commits mostra quem fez o quê — quem não contribuiu tem nota diferente dentro do grupo
- **IA é permitida e incentivada — declarando:** README diz qual ferramenta usou e como; cópia sem entendimento = zero

Rubrica — 50 pontos

Critério	Pts
Artefato técnico funciona e é reproduzível	18
Apresentação (clareza + demo ao vivo, ~10 min)	10
Relatório (evidência, honestidade sobre limites)	10
Q&A — domínio do que fez (5 min)	7
Originalidade e profundidade	5

- Com os 4 labs e a Atividade 1, o curso fecha em **100 pontos**
- Demo que falha ao vivo **não zera**: tenha o plano B (gravação/print) e explique o que deu errado

Escolhendo o tema — pela habilidade que o grupo quer mostrar

Trilha	Temas (2026)	Emulador?
🤖 **Agentes de IA**	**Playwright Agents** · **Maestro MCP** + agente de código · crítica da geração por LLM	depende
🔭 **Shift-right**	observabilidade como superfície de teste · feature flags/canary	não
🧠 **App com LLM**	*evals* e red-teaming (GitHub Models, grátis)	não
🌐 **Web / PWA**	visual regression · Lighthouse CI com budget · Service Worker/offline	não
📱 **Mobile**	Maestro avançado + POM · visual testing · flakiness	sim
🏗️ **Pipeline**	CI com sharding e quality gate · mutation testing (Stryker)	não
♿ **A11y / Perf / Seg.**	WCAG 2.2 + axe-core · Flashlight/Reassure · OWASP MASVS	depende

Exemplos de projeto — o tamanho certo

Cabe em uma semana

- 8 flows Maestro em `tags` (smoke/regressão) + relatório JUnit + CI no GitHub Actions
- Playwright: suíte de regressão visual do CineFav Web em 3 viewports + análise de falsos positivos
- axe-core no CineFav Web: lista de violações, 3 corrigidas, antes × depois

Grande demais

- "Testar tudo" em web + PWA + iOS + Android + IA
- Depender de serviço pago ou de aparelho físico que o grupo não tem

- **Pergunta-guia:** "que **risco real** do CineFav o nosso projeto cobre?"

Como será a apresentação (~10 min + 5 de Q&A)

1. **Problema** — que risco/bug o grupo quis pegar (1 min)
2. **Ferramenta** — por que esta e não outra (1 min)
3. **Demo ao vivo** — rodando, mostrando passar **e** falhar (4 min)
4. **Achado** — o que descobriram, número ou print (3 min)
5. **Limite** — o que **não** funcionou ou não cobre (1 min)

- Ordem de apresentação: **sorteio** na Aula 6
- Todos do grupo falam; pode apresentar do notebook ou de gravação, desde que tenha **demo real**

Como escolher hoje, em 3 passos

1. **Skill:** o que o grupo quer ter no portfólio? (web, mobile, pipeline, IA...)
2. **Viabilidade:** temos o que precisa **em casa**, de graça? (emulador? token? Docker?)
3. **Recorte:** descreva em **uma frase** o que vai demonstrar

> **Combinado:** hoje, ao final da aula, cada grupo manda **integrantes + tema + a frase de recorte** no Teams. Não sabe? fala com o prof — melhor decidir hoje do que descobrir no dia 28.

Cronograma até o fim

Data	O que acontece
22/10 (hoje)	Lab Mobile e Lab Playwright PWA vencem · escolha do tema
26–28/10	Vocês constroem · dúvidas por Teams / office hours
29/10 — Aula 6	**Apresentações**
Depois	Entrega: repositório + relatório (prazo no Canvas)

> Prazos reais e pontuação vivem no **Canvas** — o que está aqui é o combinado da aula.

Leituras de apoio

- **Maestro Docs** (oficial) — *Selectors* e *Best practices*
- **Maestro Docs** — receita *Page Object Model (POM)*
- **Wang et al.** (2023) e **Lam et al.** (2019, iDFlakies) — causas de testes flaky
- **Playwright Docs** — *Visual comparisons* e *Best practices*
- **W3C WCAG 2.2** — Quick Reference
- Artigos de referência do TF de *Testes de Aplicações Mobile* (repo público da disciplina)

Recap da aula — e o que vem

- **Hoje:** hierarquia de seletores, esperas, suíte organizada, Lab Arquitetura de Testes, Seminário Técnico apresentado
- **Hoje, ainda:** mande **integrantes + tema + recorte** no Teams
- **29/10 (Aula 6):** apresentações do Seminário Técnico
- **Última aula (29/10):** seminários — fim do curso

Bons estudos!

📧 jackson.96@gmail.com
🔗 linkedin.com/in/3jacksonsmith
🐙 github.com/jacksonsmith